

Spurious Rewards: Rethinking Training Signals in RLVR

Rulin Shao*, Shuyue Stella Li*, Rui Xin*, Scott Geng*, Yiping Wang, Sewoong Oh, Simon Shaolei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, Yulia Tsvetkov, Hannaneh Hajishirzi, Pang Wei Koh, Luke Zettlemoyer

University of Washington · Allen Institute for AI · UC Berkeley (equal contribution)*

RLVR works, but what is the reward actually doing?

- **RLVR** is the default recipe for improving LLM math reasoning
- Gains are implicitly credited to **correct, verifiable supervision**
- **Qwen2.5-Math** has become the de facto testbed for open RLVR research
- Yet the **mechanism behind the gains** remains poorly understood
- **Research question:** how much real signal must the reward carry?

Spurious rewards nearly match ground truth on Qwen, but not elsewhere

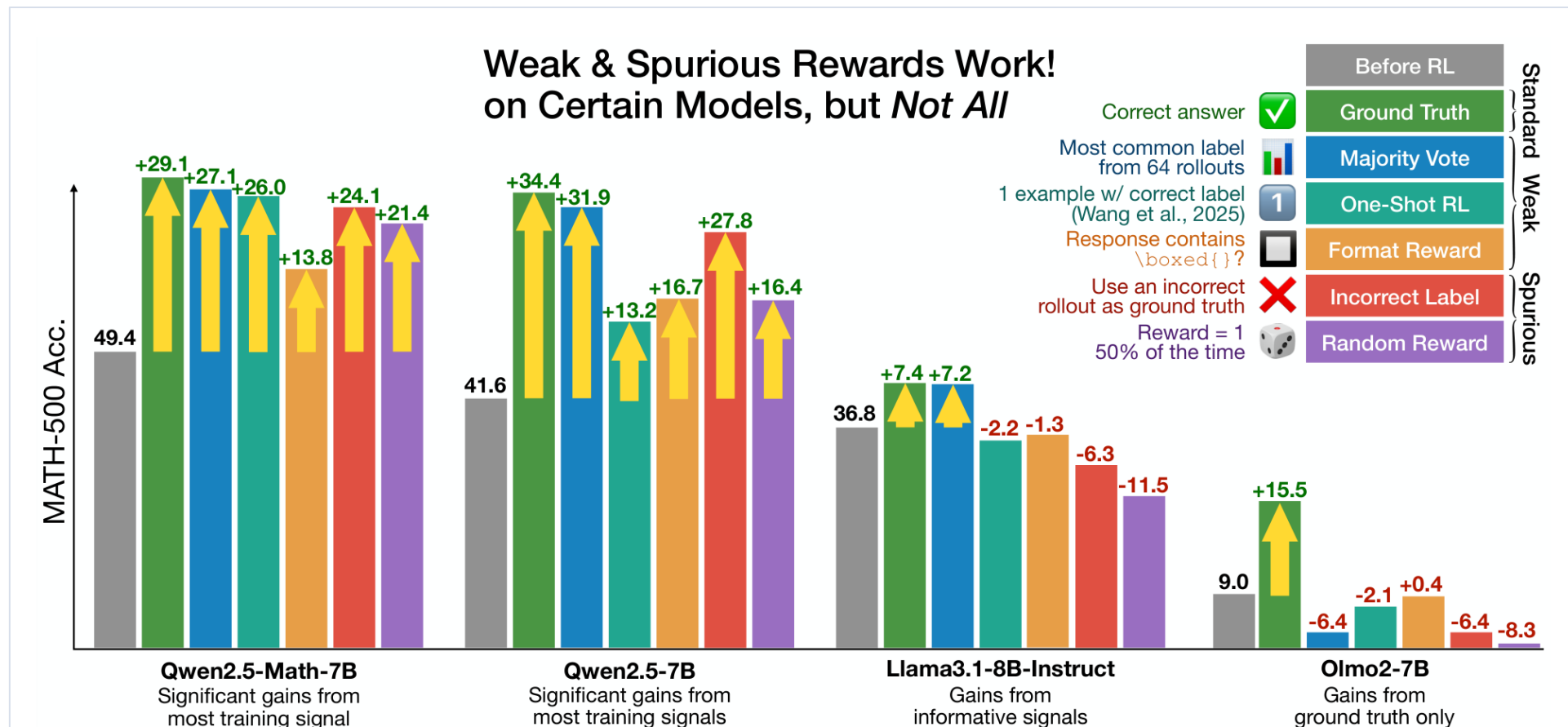


Figure 1. MATH-500 accuracy after 300 RLVR steps under each training signal (absolute gain over the pre-RL model).

A ladder of rewards: standard → weak → spurious

- **Ground truth:** reward verifiably correct answers (supervision upper bound)
- **Majority vote** (weak): the pre-RL model answers each prompt **64 times**
 - Most common final answer becomes the label, fixed offline and never refreshed
- **Format** (weak): reward any non-empty `\boxed{}`, ignoring correctness entirely
- **Random** (spurious): reward 1 with probability $\gamma = 0.5$, independent of the response
- **Incorrect** (spurious): same vote, but keep only prompts where the label is **wrong**
 - Reward that one specific wrong answer, so reward sparsity stays comparable
- **Held constant** across all five: data, model and every GRPO hyperparameter

GRPO on DeepScaleR, evaluated on standard math benchmarks

- **Algorithm:** GRPO¹ with binary 0/1 rewards; KL and entropy losses disabled ($\lambda = 0$)
- **Training data:** DeepScaleR² (~40K competition math problems, verifiable answers)
- **Models:** 10 across the Qwen2.5, Llama3 and OLMo2 families
- **Benchmarks:** MATH-500³ (pass@1), AMC and AIME⁴ 2024/2025 (avg@8)
- **Hyperparameters:** lr 5e-7, mini-batch 128, rollout batch 64, 16 rollouts/prompt, $\tau = 1$
- **Compute:** ~24 h on 8xA100 per RLVR run; curves reported to 300 steps

Every reward, even random, lifts Qwen2.5-Math within ~50 steps

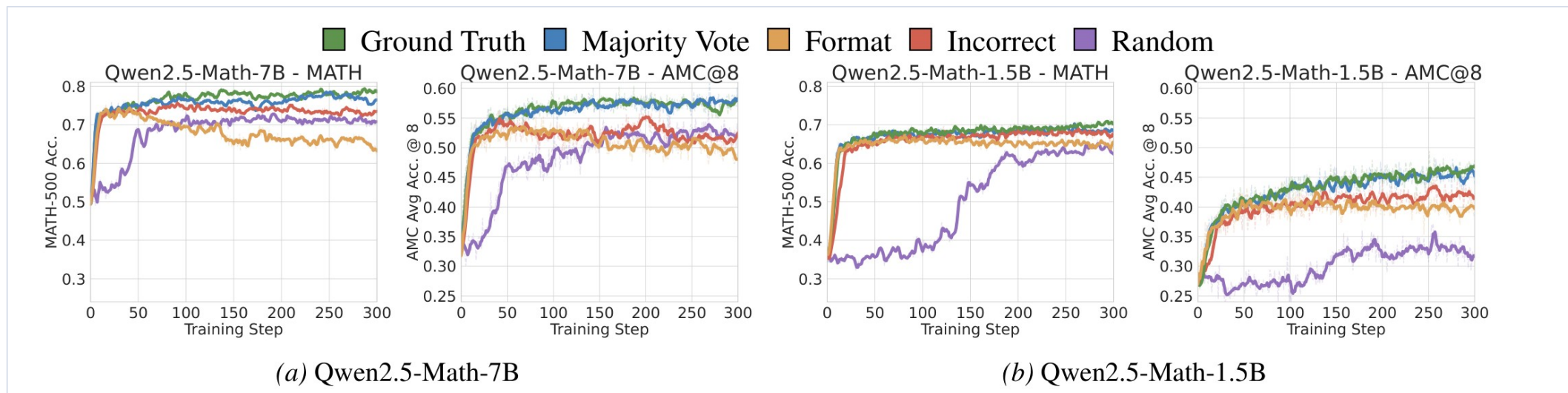


Figure 2. MATH-500 (pass@1) and AMC (avg@8) across 300 RLVR steps; smoothed over a window of 10, dotted lines unsmoothed.

- All rewards, including pathological ones, improve within the **first 50 steps**
- **Incorrect labels: +24.1** on MATH-500 vs **+29.1** for ground truth (7B)
- **Random rewards: +21.4** on the 7B, but slower on the 1.5B: **~100 steps**
- Same picture on AMC; AIME24: format **+10.3** vs ground truth **+15.3**

Spurious rewards rarely help anything that is not Qwen

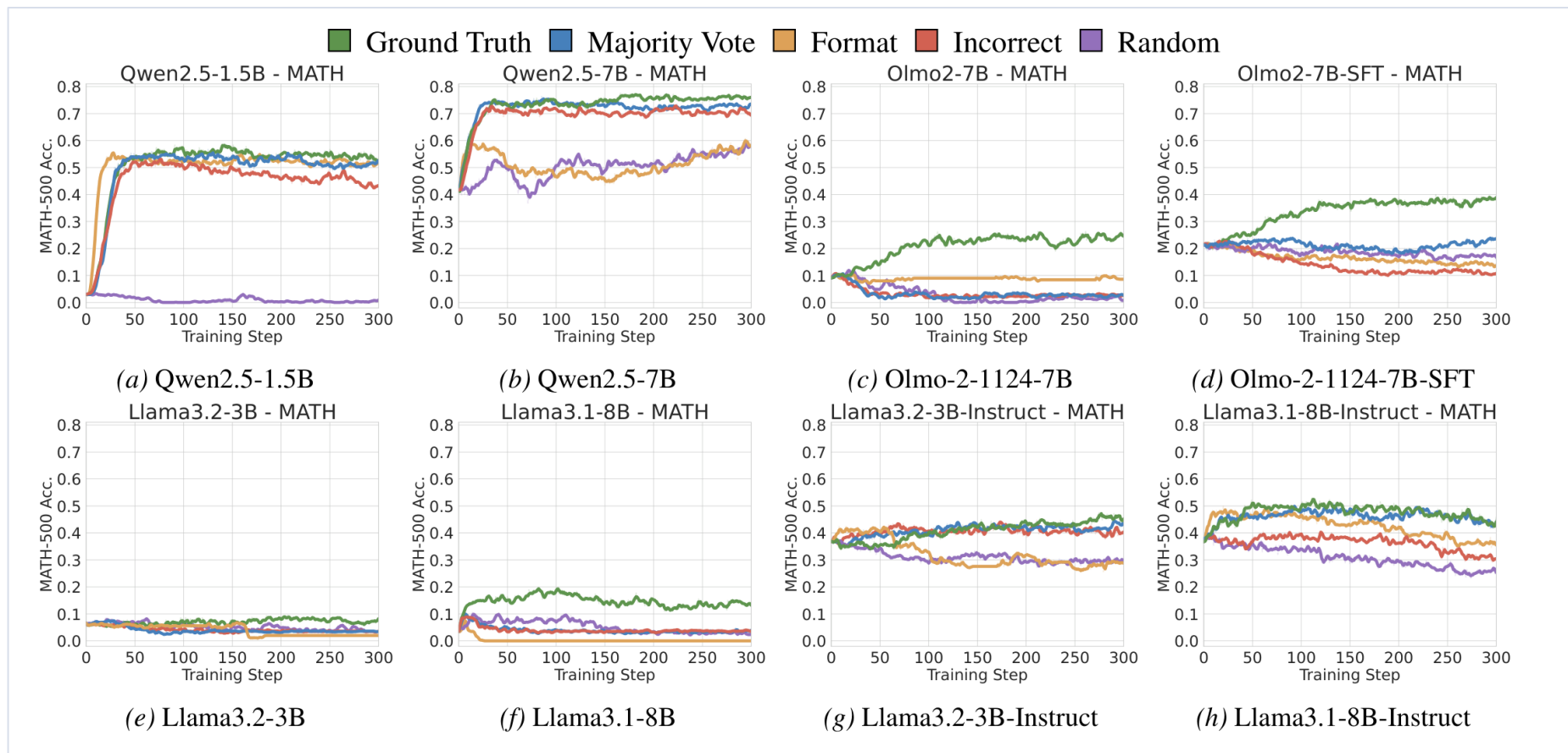


Figure 3. Varying rewards across eight additional models on MATH-500. Gains on non-Qwen2.5 families are substantially smaller.

The practical warning: validate RLVR beyond Qwen

- Every weak or spurious reward **fails to help at least one other model**
- Qwen appears **unusually robust to reward strength**; family behaviour is consistent
- Divergence suggests **pretraining priors, not the reward, shape RL dynamics**
- Re-ran **TTRL**¹ and **One-Shot RL**²: strong on Qwen, often flat elsewhere (App. E)
- Models that already went through **RL post-training gain little** from any reward (App. J)

Why do random rewards train anything at all? Clipping.

- Careful: the reward is **not** zero-mean, since $\mathbf{E}[r] = \gamma = 0.5$
- **Zero-mean advantage is by construction** in GRPO, equally true of ground truth
- What vanishes is the **expected gradient**, because advantage is **independent** of the rollout
- Each single step is still a **full-size kick**: noise, not smallness
- Rollouts are reused for **8 gradient updates**, so the ratio drifts off 1 and clip fires
- Kill clipping three ways → gains **vanish** (two variants also cut updates 8×)

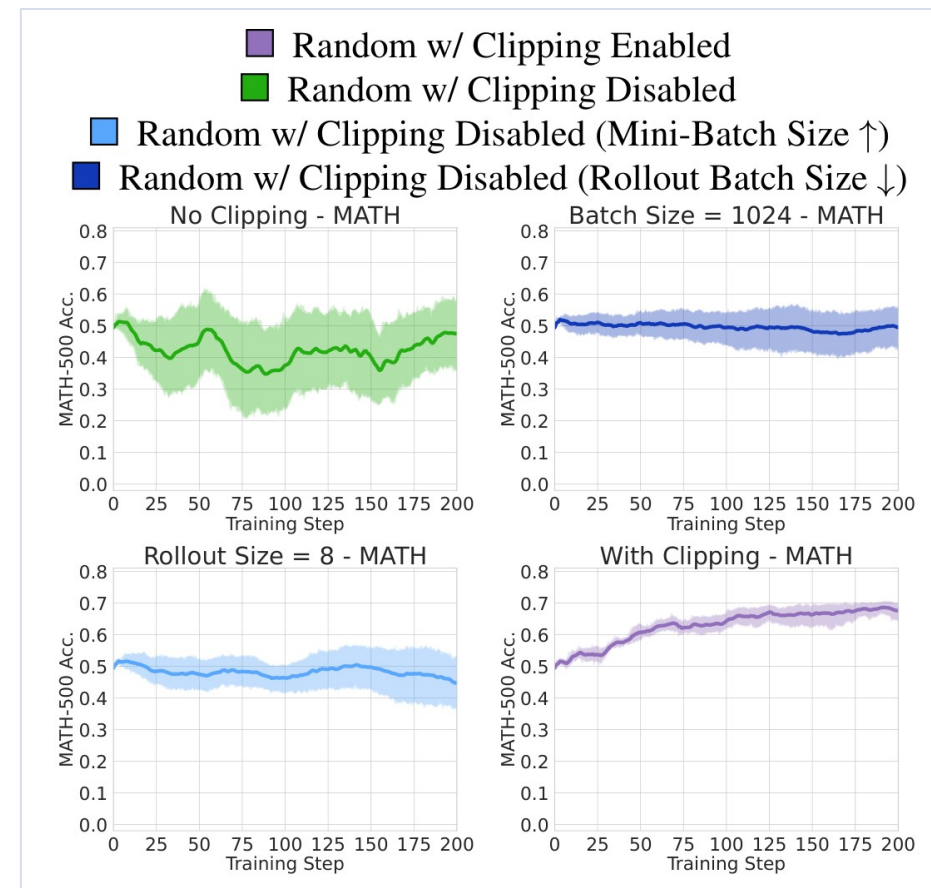


Figure 4. Averaged over seeds, Qwen2.5-Math-7B.

Eq. 1: what GRPO actually maximises

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\text{old}}(\cdot|x)} \left[\sum_{t=1}^{|y|} \min \left(\rho_t(y; \theta) \hat{A}(x, y), \right. \right. \quad (1) \\ \left. \left. \text{clip}(\rho_t(y; \theta), 1 - \epsilon_c, 1 + \epsilon_c) \hat{A}(x, y) \right) \right],$$

Eq. 1, with the clipping term in red. The KL penalty is omitted because it is disabled in these experiments.

- **ρ (rho)** is the importance ratio: current policy probability over behavior policy
- **Â** is the advantage, centred within each prompt, so it averages to zero by construction
- The **min** picks whichever of the two products is smaller, which is PPO's pessimistic bound
- Everything downstream comes from **when that min selects the clipped branch**

Clipping is a sign filter, not a source of signal

	Ratio below the band ($< 1 - \epsilon$)	Ratio above the band ($> 1 + \epsilon$)
Advantage > 0 · this rollout pushes the token up	not clipped → push up survives	<i>clipped</i> → <i>gradient 0</i>
Advantage < 0 · this rollout pushes the token down	<i>clipped</i> → <i>gradient 0</i>	not clipped → push down survives

- Inside the band **nothing is clipped**: both signs flow and **cancel**
- So the 0 in Eq. 2 is **cancellation, not suppression**, the usual misreading
- Outside the band one sign survives, but alone that is only a **restoring force**

The clip band is relative, but the probability ceiling is absolute

- **High-prior token:** behavior-policy probability **0.85**
 - Getting clipped upward needs $0.85 \times 1.2 = \mathbf{1.02}$
 - Probabilities cap at 1, so that cell is **unreachable**
 - Only **0 or push up** remain, so it can **never be pushed down**
 - Every token above $1/(1+\epsilon) \approx \mathbf{0.833}$ is in this regime
 - Frequently sampled tokens **drift up for free**
- **Low-prior token:** behavior-policy probability **0.02**
 - The same threshold sits at just **0.024**
 - Only 0.004 of headroom, so it is **clipped constantly**
 - All three cells stay live, so **push down dominates**
 - The restoring force **loses its downward half** → a ratchet
 - Rare tokens are **pushed down by pure noise**

Eq. 2: the expected gradient under random rewards

$$\mathbb{E}[\nabla_{\theta} J(\theta)] \propto \mathbb{E}_{x,y} \left[\begin{cases} \nabla_{\theta} \log \pi_{\theta,x}(y_t), & R_{\theta} < 1 - \epsilon_c, \\ 0, & |R_{\theta} - 1| \leq \epsilon_c, \\ -\nabla_{\theta} \log \pi_{\theta,x}(y_t), & R_{\theta} > 1 + \epsilon_c, \end{cases} \right], \quad (2)$$

Eq. 2, with R = current over behavior policy probability for that token. Reward-independent scalars are factored out.

- Three branches: **below the band, inside it, above it**
- Below pushes the token **up**, above pushes it **down**: a restoring force
- Inside the band the expectation is **0**, because the two signs cancel
- The reward **does not appear**: it integrated out, leaving only the ratio R

The full chain, and the link that decides the outcome

- 1. Each rollout batch is reused for **8 gradient updates** → the ratio leaves 1
- 2. Clipping **filters by sign** → a residual gradient appears out of pure noise
- 3. The **ceiling at 1** makes that filter one-sided for high-prior tokens
- 4. Probability mass **sharpens onto the model's own prior**, no new ability
- 5. Accuracy rises **only if that prior is already correct**, so it is model-dependent
- Links **1 to 4 are universal**; link **5** is where Qwen and Llama part ways

If the reward taught nothing, what got amplified?

- Sharpening is **sign-neutral**, so everything rides on **what sits at the peak**
- **Central hypothesis:** RLVR outcomes differ because **pretraining strategies** differ
- Some strategies are **readily amplified**, others are hard to elicit or **absent**
- So Qwen2.5-Math should hold a prior that is **amplifiable and correlated with correctness**
- Llama and OLMo should either **lack one** or hold one that **does not help**
- Next: **code reasoning**, a controlled and observable instance of such a strategy

Code reasoning: a strong pre-existing Qwen-Math behavior

- Qwen2.5-Math-7B writes Python in **65.0%** of MATH-500 answers
- It has **no code interpreter**, so the output is autoregressively hallucinated
- Yet code answers are **far more accurate** than natural-language ones
- Other families are **No-Code** (Llama, Qwen2.5-1.5B, OLMo2-7B)...
- ...or **Bad-Code** (OLMo2-7B-SFT, Qwen2.5-7B): lots of code, worse accuracy

MATH Question:

What is the distance, in units, between the points $(2, -6)$ and $(-4, 3)$? Express your answer in simplest radical form.

Qwen2.5-Math-7B Solution (correct):

To find the distance between two points (x_1, y_1) and (x_2, y_2) in a Cartesian plane...

Let's break this down step-by-step and compute the result using Python.

```
1 import math
2 ...
3 # Calculate the distance using the
  distance formula
4 distance = math.sqrt(dx**2 + dy**2)
5 print(distance)
```

output: 10.816653826391969

...

Thus, the final answer is: $3\sqrt{13}$

Figure 5. Code reasoning example.

Code helps Qwen-Math and hurts everyone else

Model	Qwen2.5-Math-7B	Qwen2.5-Math-1.5B	Qwen2.5-7B	OLMo2-7B-SFT
Code Frequency	65.0	53.6	92.2	98.0
Acc. w/ Code	60.9	52.6	39.9	21.0
Acc. w/ Lang	35.0	17.2	61.5	40.0

Table 1. Fraction of MATH-500 responses containing Python before RL, with accuracy split by response type.

- Qwen2.5-Math-7B: **60.9 with code vs 28.0 without**, a **33-point** gap
 - The table's 35.0 fails the mixture check: $0.65 \times 60.9 + 0.35 \times 35.0 = 51.8$, not 49.4
- Qwen2.5-7B and OLMo2-7B-SFT write **more** code (92.2 / 98.0) but do **worse** with it
- Effective code reasoning is a **distinctive Qwen2.5-Math prior**, present before RL
- Qwen2.5-1.5B, OLMo2-7B and both Llama-Instruct models **never generate code**

Spurious rewards buy accuracy by buying code reasoning

- Under weak/spurious rewards code frequency jumps to **~90% within 15 steps**
- It reaches **95.6% for random rewards**, tracking the accuracy curve closely
- With **ground truth**, code frequency rises then **declines** as language reasoning improves
- For **Bad-Code** models, gains instead correlate with **less** code
- 58.3%** of the 7B's gain comes from problems switching **Lang→Code** (App. H)

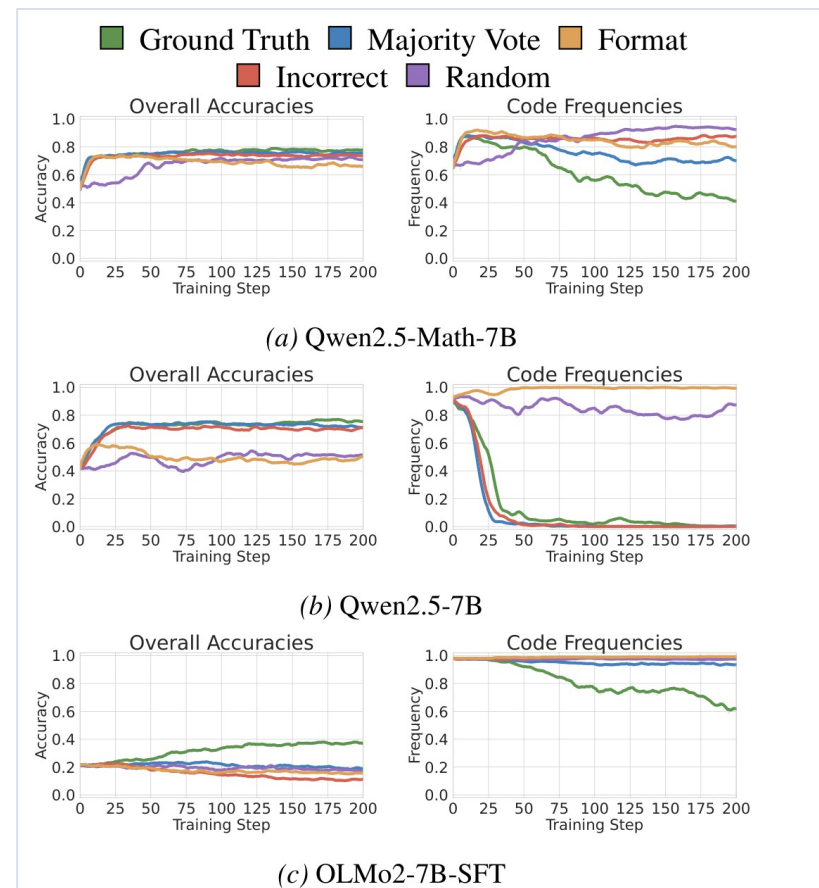


Figure 6. Accuracy (left) and Python-code frequency (right).

Forcing code reasoning reproduces the effect without any RL

Model	Original	Prompting	Abs. Diff.
Qwen2.5-Math-1.5B	36.2%	60.4%	+24.2%
Qwen2.5-Math-7B	49.4%	64.4%	+15.0%
Qwen2.5-1.5B	3.0%	13.0%	+10.0%
Qwen2.5-7B	41.6%	22.2%	-19.4%
Llama3.2-3B-Instruct	36.8%	8.2%	-28.6%
Llama3.1-8B-Instruct	36.8%	15.2%	-21.6%
OLMo2-7B	9.0%	7.8%	-1.2%
OLMo2-7B-SFT	21.4%	18.6%	-2.8%

Table 2. MATH-500 accuracy when the first generated sentence is forced to be “Let's solve this using Python.”

- Qwen2.5-Math-1.5B **+24.2**, Qwen2.5-Math-7B **+15.0**, Qwen2.5-1.5B **+10.0**
- Llama3.2-3B-Instruct **-28.6**, Llama3.1-8B-Instruct **-21.6**, Qwen2.5-7B **-19.4**
- Sign of the effect is predicted by the **pre-existing code ability** in Table 1

A reward for the string “python” behaves like a real reward

- Reward = 1 iff the response **contains the string “python”**, nothing else
- Qwen2.5-Math-7B exceeds **99% code reasoning within 20 steps**
- Accuracy gains appear **primarily in Qwen2.5-Math** models
- RLVR induction **matches or exceeds** the prompting intervention there
- All other families show **limited or negative** movement

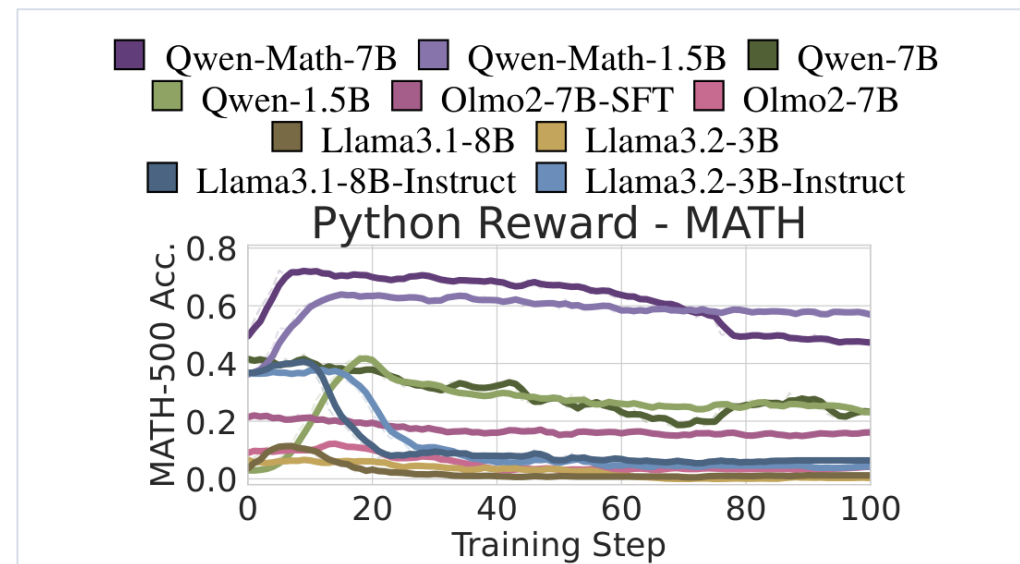


Figure 7. Python reward, MATH-500.

Conclusions

- **Reward correctness is not what drove RLVR gains** on Qwen2.5-Math
- **GRPO's clipping term** amplifies high-prior tokens even under pure noise
- Same loss, same sign: ground truth encodes **solution quality**, noise encodes **prior position**
- Gains come from **eliciting pre-existing behaviors**, here code reasoning
- **Pretraining priors decide the outcome**: Qwen $\neq$ Llama $\neq$ OLMo
- **Always run random / format reward baselines** and ≥ 2 model families
- Spurious rewards are a **diagnostic**, never a training recipe

Future work

- **Measure pass@k directly:** the elicitation claim is inherited^{1,2}, not tested here
- **Look past code reasoning:** repetition is a second elicitable pattern (App. G)
- **Isolate clipping cleanly:** two of three ablations also cut updates 8× (Fig. 10)
- **Separate prompt from reward:** a LIPSUM prompt alone buys +19.4 (Table 5)
- **Scale up:** does the account hold past 8B parameters and 300 steps?
- **Move beyond math:** every result here is on math benchmarks